



Tableaux

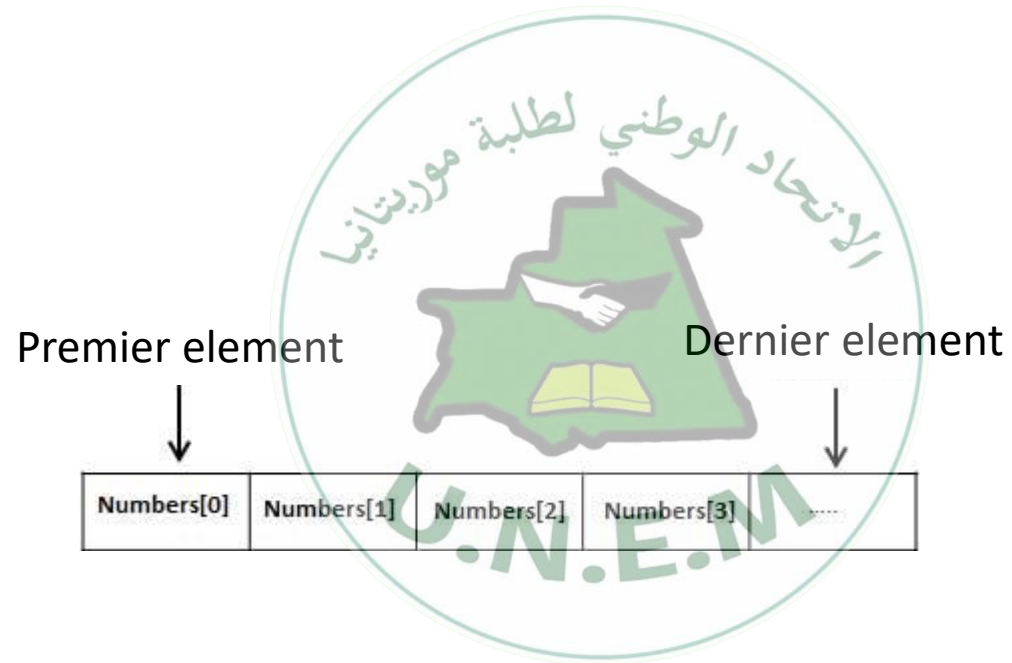


Tableaux

- Tableaux une sorte de structure de données qui peut stocker une collection séquentielle de taille fixe d'éléments du même type. Un tableau est utilisé pour stocker une collection de données, mais il est souvent plus utile de considérer un tableau comme une collection de variables du même type.
- Au lieu de déclarer des variables individuelles, telles que nombre0, nombre1, ... et nombre99, vous déclarez une variable tableau telle que nombres et utilisez nombres[0], nombres[1] et ..., nombres[99] pour représenter variables individuelles.
- Un élément spécifique dans un tableau est accessible par un index.
- Tous les tableaux sont constitués d'emplacements de mémoire contigus. L'adresse la plus basse correspond au premier élément et l'adresse la plus haute au dernier élément.



Tableaux





déclarer des tableaux

- Pour déclarer un tableau en C, un programmeur spécifie le type des éléments et le nombre d'éléments requis par un tableau comme suit –

```
type arrayName [ arraySize ];
```

- C'est ce qu'on appelle un tableau unidimensionnel. arraySize doit être une constante entière supérieure à zéro et type peut être n'importe quel type de données C valide. Par exemple, pour déclarer un tableau de 10 éléments appelé balance de type double, utilisez cette instruction –

```
double balance[10];
```

- Ici, balance est un tableau de variables suffisant pour contenir jusqu'à 10 nombres doubles.



initialisation des tableaux

- Vous pouvez initialiser un tableau en C un par un ou en utilisant une seule instruction comme suit –

```
double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

- Le nombre de valeurs entre accolades { } ne peut pas être supérieur au nombre d'éléments que nous déclarons pour le tableau entre crochets [].
- Si vous omettez la taille du tableau, un tableau juste assez grand pour contenir l'initialisation est créé. Donc, si vous écrivez –

```
double balance[] = {1000.0, 2.0, 3.4, 7.0, 50.0};
```

- Vous allez créer exactement le même tableau que dans l'exemple précédent. Voici un exemple pour affecter un seul élément du tableau -

```
balance[4] = 50.0;
```

	0	1	2	3	4
balance	1000.0	2.0	3.4	7.0	50.0



Accéder aux éléments du tableau

- Un élément est accessible en indexant le nom du tableau. Cela se fait en plaçant l'index de l'élément entre crochets après le nom du tableau. Par exemple :

```
double salary = balance[9];
```

```
#include <stdio.h>
int main () {
int n[ 10 ];          /* n est un tab de 10 entiers */
int i,j;              /* initialization des elements du tab*/
for ( i = 0; i < 10; i++ ) {
    n[ i ] = i + 100;
}
for (j = 0; j < 10; j++ ) {
    printf("Element[%d] = %d\n", j, n[j] );
}
return 0;
}
```

```
Element[0] = 100
Element[1] = 101
Element[2] = 102
Element[3] = 103
Element[4] = 104
Element[5] = 105
Element[6] = 106
Element[7] = 107
Element[8] = 108
Element[9] = 109
```



Tableaux avec les Fonctions





Comment passer des tableaux à une fonction ?

- Un nom de tableau peut être utilisé comme argument d'une fonction :
 - Autorise le passage du tableau entier à la fonction.
 - La façon dont elle est transmise diffère de celle des variables ordinaires.
- Règles :
 - Le nom du tableau doit apparaître seul en argument, sans indices.
 - L'argument formel correspondant s'écrit de la même manière.
 - Déclaré en écrivant le nom du tableau avec une paire de crochets vides.



Comment passer des tableaux à une fonction ?

On peut aussi écrire

```
float x[100] ;
```

Mais la façon dont la fonction est écrite la rend générale ; cela fonctionne avec des tableaux de n'importe quelle taille.

```
float average(a,x[]){  
    int a;  
    float x[];
```

```
    .  
    .  
    .  
}
```

```
int main(){  
    int n;  
    float list[100], avg;  
    .  
    .  
    avg = average(n,list);  
    .  
    .  
}
```



Passage de tableaux en tant qu'arguments de fonction

```
#include <stdio.h>
```

```
double getAverage(int arr[], int size) {  
    int i;  
    double avg;  
    double sum = 0;  
    for (i = 0; i < size; ++i) {  
        sum += arr[i];  
    }  
    avg = sum / size;  
    return avg;  
}
```

```
int main () {  
    int balance[5] = {1000, 2, 3, 17, 50};  
    double avg;  
    avg = getAverage( balance, 5 );  
    printf( "Average value is: %f ", avg );  
    return 0;  
}
```

Average value is: 214.400000



Example 1:

```
#include <stdio.h>

double getAverage(int arr[], int size){
    int i;
    double avg;
    double sum;
    for (i = 0; i < size; ++i){
        sum += arr[i];
    }
    avg = sum / size;
    return avg;
}

int main (){
    int balance[5] = {1000, 2, 3, 17, 50};
    double avg;
    avg = getAverage( balance, 5 ) ;
    printf( "Average est: %f ", avg );
    return 0;
}
```



Exemple 2:

```
int mini(int x[], int size){
    int i, min=4;

    for(i=0;i<size;i++){
        if(min>x[i]){
            min=x[i];
        }
    }
    return min;
}

int main(){
    int a[100], i, n;

    scanf("%d", &n);

    for(i=0;i<n;i++){
        scanf("%d",&a[i]);
    }

    printf("le min est %d", mini(a,n));
}
```



Tableaux multi-dimensionnels





Tableaux multi-dimensionnels

- Le langage de programmation C autorise les tableaux multidimensionnels. Voici la forme générale d'une déclaration de tableau multidimensionnel –

```
type name[size1][size2]...[sizeN];
```

- Par exemple, la déclaration suivante crée un tableau d'entiers tridimensionnel –

```
int threedim[5][10][4];
```



Tableaux bidimensionnels

- La forme la plus simple de tableau multidimensionnel est le tableau à deux dimensions. Un tableau à deux dimensions est, par essence, une liste de tableaux à une dimension.
- Pour déclarer un tableau d'entiers à deux dimensions de taille [x] [y], vous écririez quelque chose comme suit –

```
type arrayName [ x ][ y ];
```

- Où type peut être n'importe quel type de données C valide et arrayName sera un identifiant C valide. Un tableau à deux dimensions peut être considéré comme un tableau qui aura x nombre de lignes et y nombre de colonnes. Un tableau à deux dimensions a, qui contient trois lignes et quatre colonnes peut être représenté comme suit –

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

- Ainsi, chaque élément du tableau a est identifié par un nom d'élément de la forme a[i][j], où 'a' est le nom du tableau, et 'i' et 'j' sont les indices qui identifient de manière unique chaque élément dans 'a'.



Initialisation de tableaux à deux dimensions

- Les tableaux multidimensionnels peuvent être initialisés en spécifiant des valeurs entre parenthèses pour chaque ligne. Voici un tableau avec 3 lignes et chaque ligne a 4 colonnes.

```
int a[3][4] = { {0, 1, 2, 3} , /* initialisation de la ligne indexée par 0 */  
               {4, 5, 6, 7} , /* initialisation de la ligne indexée par 1 */  
               {8, 9, 10, 11} /* initialisation de la ligne indexée par 2 */  
};
```

- Les accolades imbriquées, qui indiquent la ligne voulue, sont facultatives. L'initialisation suivante est équivalente à l'exemple précédent -

```
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```




Accéder aux éléments de tableau à deux dimensions

- Un élément dans un tableau à deux dimensions est accessible en utilisant les indices, c'est-à-dire l'index de ligne et l'index de colonne du tableau. Par exemple –

```
int val = a[2][3];
```

- L'instruction ci-dessus prendra le 4ème élément de la 3ème ligne du tableau. Vous pouvez le vérifier dans la figure ci-dessus. Vérifions le programme suivant où nous avons utilisé une boucle imbriquée pour gérer un tableau à deux dimensions -

```
#include <stdio.h> int main () { /* un tableau de 5 lignes et 2 colonnes*/  
int a[5][2] = { {0,0}, {1,2}, {2,4}, {3,6},{4,8}};  
int i, j; /* valeurs de chaque élément du tableau */  
for ( i = 0; i < 5; i++ ) {  
    for ( j = 0; j < 2; j++ ) {  
        printf("a[%d][%d] = %d\n", i,j, a[i][j] );  
    }  
}  
return 0;  
}
```



Accéder aux éléments de tableau à deux dimensions

```
#include <stdio.h> int main () { /* un tableau de 5 lignes et 2 colonnes*/  
int a[5][2] = { {0,0}, {1,2}, {2,4}, {3,6},{4,8}};  
int i, j; /* valeurs de chaque élément du tableau */  
for ( i = 0; i < 5; i++ ) {  
    for ( j = 0; j < 2; j++ ) {  
        printf("a[%d][%d] = %d\n", i, j, a[i][j] );  
    }  
}  
return 0;  
}
```

- Lorsque le code ci-dessus est compilé et exécuté, il produit le résultat suivant -

```
a[0][0]: 0  
a[0][1]: 0  
a[1][0]: 1  
a[1][1]: 2  
a[2][0]: 2  
a[2][1]: 4  
a[3][0]: 3  
a[3][1]: 6  
a[4][0]: 4  
a[4][1]: 8
```